

Final Project Report

Ha Long Truong

ASU Ira A. Fulton Schools of Engineering

1. Reading the maze from camera

We connected the camera with our laptop and used OpenCV to program the camera to capture the image with the maze in it.

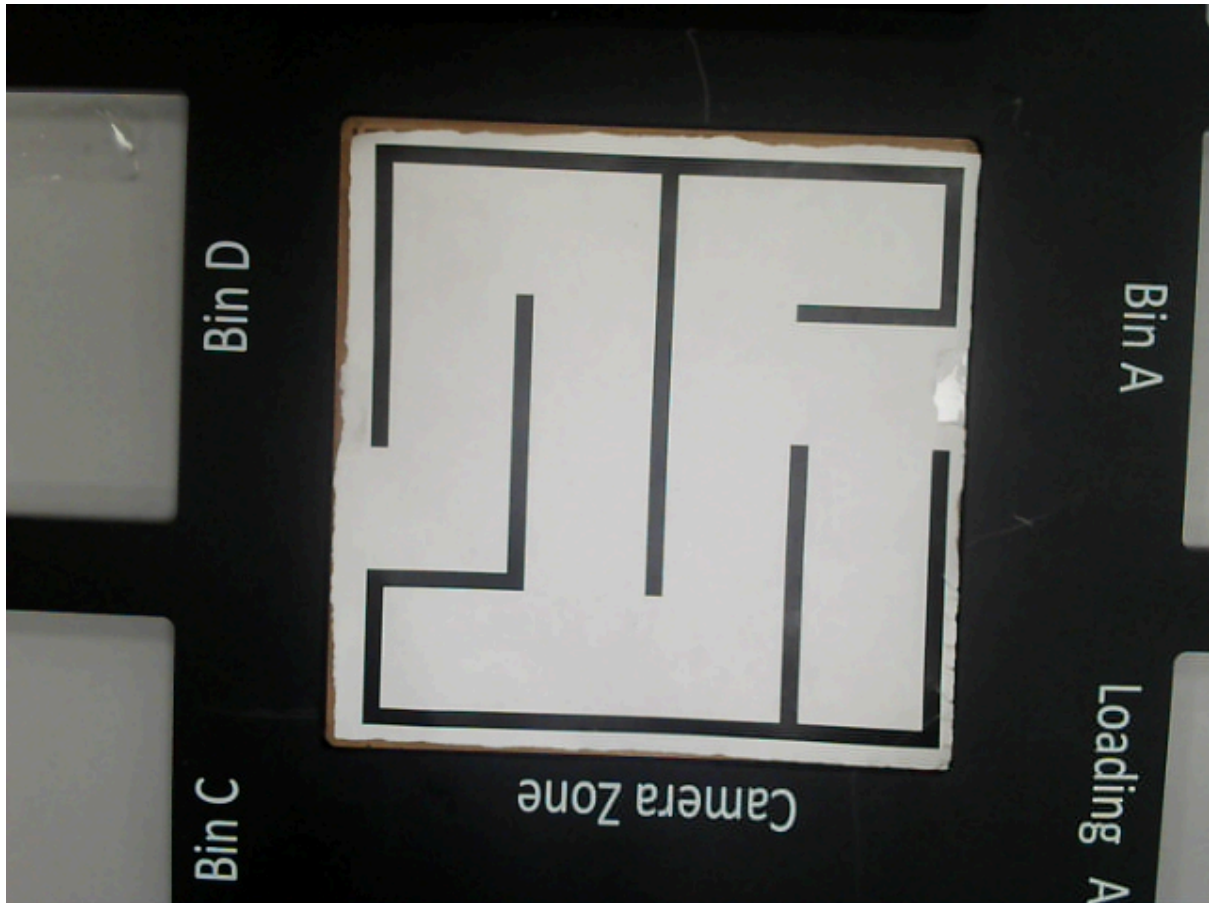


Figure 1: Image captured from camera

Then, we edited the image by cropping and rotating to extract the image. We also needed to turn the maze image into a black and white image. We used a python code to extract the maze, or we could also use a photo editing app to directly edit the image.

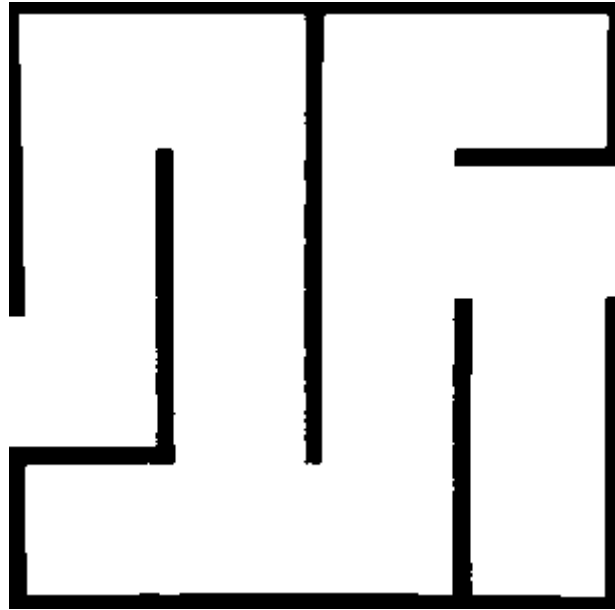


Figure 2: Maze extracted from the image

2. Solving the maze

To solve the maze, we decided to divide the maze into 16 square areas. OpenCV can determine the wall and the path based on the black and white color. The idea is to check which square that we can move to from our current square. We used bit masking for this step. We set 1 (2^0), 2 (2^1), 4 (2^2), 8 (2^3) for top, right, left, bottom respectively if we do not find any wall in those directions. We add those numbers together and give that value to the current square. For example, if a square has a value of 10 ($2^1 + 2^3$), we know that we can move right and bottom while we can not move top and left. By doing this for all 16 squares, we can create a matrix that represents our maze in our code. Then, we create two functions. One is to check our current position and determine the next possible path that we can take. The other one is to recursively call the first function until we reach the end goal. By doing this we can find out the path to go from our start point to the end point.

```
Possible moves from (2, 0): [(2, 1)]
Possible moves from (2, 1): [(2, 2), (2, 0)]
Possible moves from (2, 2): [(1, 2), (2, 1)]
Possible moves from (1, 2): [(2, 2), (1, 1)]
Possible moves from (1, 1): [(1, 2), (1, 0)]
Possible moves from (1, 0): [(0, 0), (1, 1)]
Possible moves from (0, 0): [(0, 1), (1, 0)]
Possible moves from (0, 1): [(0, 2), (0, 0)]
Possible moves from (0, 2): [(0, 3), (0, 1)]
Possible moves from (0, 3): [(1, 3), (0, 2)]
Path found: [(2, 0), (2, 1), (2, 2), (1, 2), (1, 1), (1, 0), (0, 0), (0, 1), (0, 2), (0, 3), (1, 3)]
```

Figure 3: Solving the maze with start at point (2, 0) and end at point (0, 3)

3. Moving the robot

We recorded the positions of the 16 squares that we used and ran inverse kinematics beforehand. We used Matlab Inverse Kinematics Designer to solve this problem. We loaded the robots into Matlab, chose the BFGS Gradient

solver algorithm, and entered our desired positions and orientations. We also assigned this information to their respective squares in the maze matrix. Thus, when we found out the path to solve the maze, we also found out how we wanted our robot to move. We looped through our list of path squares and sent the joint angles from the inverse kinematics result to the robot. The commands are sent in the form of messages via Python sockets.